

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

Python Fundamentals Masterclass

Course Code: C179

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 1.0

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	4 October 2026	First release of the Python Fundamentals Masterclass Learner Guide — detailed step-by-step for all 21 hands-on activities across six topics.	Dr Alfred Ang

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

TABLE OF CONTENTS

Introduction **4**
Course Learning Outcomes **4**
Before You Start — Environment Setup **4**
Topic 01 — Get Started on Python **5**
Activity 1 — Your First Python Script 5
Topic 02 — Data Types **6**
Activity 2 — Format Numbers 6
Activity 3 — Format String 7
Activity 4 — Split and Join Strings 8
Activity 5 — List Operations 9
Activity 6 — Set — Count Unique Vocabulary 10
Topic 03 — Operators **10**
Activity 7 — Operators 11
Topic 04 — Control Structures, Loop and Comprehension **12**
Activity 8 — Conditional — Income Classifier 12
Activity 9 — Fibonacci Sequence & Golden Ratio (While Loop) 13
Activity 10 — For Loop — Square Numbers 14
Activity 11 — Loop Multiple Sequences with zip() 15
Activity 12 — List Comprehension 16
Topic 05 — Function **17**
Activity 13 — Function — Future Value Calculator 17
Activity 14 — Multiple Return Values — Discount & Tax 18
Activity 15 — Variable Arguments — Minimum Finder 19
Activity 16 — Lambda Function 20
Activity 17 — Map 20
Activity 18 — Filter 21
Topic 06 — Modules & Packages **22**
Activity 19 — Install Third-Party Packages 22
Activity 20 — Import & Visualize Stock Data from CSV 23
Activity 21 — Import Data from Yahoo Finance 24
Keep Practising **25**
Glossary **25**

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Introduction

This Learner Guide accompanies the course Python Fundamentals Masterclass (C179), conducted by Tertiary Infotech Academy Pte Ltd. It provides the detailed step-by-step instructions for all 21 hands-on activities, organised by the six course topics. The course slides stay visual and high-level; THIS guide is where every activity is worked through line by line, so use it during class and for practice afterwards.

Each activity opens with a one-line focus statement describing the skill it practises. The guide, the slides and the lesson plan are generated from one content source, so they always stay aligned.

Course Learning Outcomes

- LO1: Apply Python programming to meet business requirements
- LO2: Code data types and operators
- LO3: Organize codes using conditional and loop for problem-solving
- LO4: Code Python functions and scripts for business use cases
- LO5: Code, import and prepare finance data using Python packages
- LO6: Test Python codes with aggregation and visualization
- LO7: Improve and document Python codes to analyse finance data

Before You Start — Environment Setup

What you need

- A laptop with a modern browser — Google Colab (https://colab.research.google.com/) runs Python in the cloud with nothing to install, and is the recommended environment for this class.
- OR a local Python 3 installation from https://www.python.org/downloads/ (tick 'Add Python to PATH' on Windows).
- Optionally an IDE: Visual Studio Code, PyCharm, Jupyter Notebook / JupyterLab, or the Anaconda distribution (https://www.anaconda.com/download/).
- For Topic 6: the packages numpy, pandas, matplotlib and yfinance (pre-installed in Colab except yfinance — install it with pip).

Verify your environment

Open a terminal (or a Colab cell — prefix shell commands with !) and confirm Python runs:

```
python --version        # or: python3 --version

python -c "print('Hello World!')"
```

Conventions used in every activity

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Code blocks are Python unless stated otherwise; type them into a Colab cell or your IDE and run with Shift+Enter.
- Predict the output before you run each cell — then compare. That is where the learning happens.
- Each activity ends with a 'Check your result' — verify it before moving on.
- The same activities live in the labs/ folder of the course repository as individual Markdown files.

Topic 01 — Get Started on Python

Overview of Python · Install Python · Python IDE · First Script · Comments.

Key concepts

- Powerful & multi-purpose — Python is used in data science, AI, finance, web and automation.
- Simple, readable syntax — Easy to learn — the ideal first programming language.
- Interpreted language — Code runs line by line — no compile step needed.
- Object-oriented — Everything in Python is an object.

Activity 1 — Your First Python Script

Focus: set up a Python environment and run a first script.

Goal: Python coding is a lot of fun and easy to start. Set up your IDE (Google Colab, Visual Studio Code or another IDE), run your first Python script and add comments.

What you'll build

A working Python environment and a script that prints output with comments. (Tools: Google Colab / VS Code, Python 3.)

Step-by-step

1. Open Google Colab at https://colab.research.google.com/ and sign in with a Google account (or open VS Code / your preferred IDE with Python 3 installed).

2. Create a new notebook: File ▸ New notebook (or create a new file test.py in your IDE).

3. Type your first Python statement in the cell and run it with Shift+Enter.

```
print("Hello World!")
```

4. Print something fun of your own — Python strings can repeat and join.

```
print("Python is fun! " * 3)

print("I am learning " + "Python")
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

5. Add a single-line comment and a multi-line comment above your code — Python ignores them but your teammates will thank you.

```
# my first Python script

"""

Author: <your name>

Date: today

"""

print("Comments added!")
```

6. If you are in an IDE, save the file as test.py and run it from the terminal.

```
python test.py
```

Check your result

The notebook / terminal prints your messages exactly once per print() call, and the commented lines produce no output.

Note: This activity is also available as labs/lab-01-your-first-python-script.md in the course repository.

Topic 02 — Data Types

Number · String · List · Tuple · Dictionary · Set.

Key concepts

- Six major data types — Number, String, List, Tuple, Dictionary and Set.
- Mutable vs immutable — Lists change in place; tuples and strings never change.
- Formatting output — format() and f-strings shape numbers and text for reports.
- Build on the basics — Stacks, queues, trees and graphs are built from these six.

Activity 2 — Format Numbers

Focus: format numeric output with f-strings.

Goal: Given a = 4.444, b = 5.555 and c = 6.6666, use the string-literal (f-string) method to format the output as: 5.55 + 6.67 + 4.4 = 17

What you'll build

An f-string that rounds each number to the required decimal places. (Tools: Python 3 (Colab / IDE).)

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Step-by-step

1. Create the three variables and their total.

```
a = 4.444

b = 5.555

c = 6.6666

d = a + b + c
```

2. Print the sum with an f-string, choosing the decimal places for each value: b to 2 dp, c to 2 dp, a to 1 dp and the total to 0 dp.

```
print(f"{b:.2f} + {c:.2f} + {a:.1f} = {d:.0f}")
```

3. Run the cell and compare with the expected output.

4. Experiment: change the format specs (e.g. {a:10.3f}) and observe the width/decimals.

```
print(f"{a:10.3f}")
```

Check your result

The output is exactly: 5.55 + 6.67 + 4.4 = 17

Note: This activity is also available as labs/lab-02-format-numbers.md in the course repository.

Activity 3 — Format String

Focus: manipulate and format strings with methods and f-strings.

Goal: Given bs = 'balance sheet' and income = 'income statement', print: "The finance statements typically require Balance sheet and INCOME STATEMENT." using the string-literal (f-string) method.

What you'll build

An f-string combining capitalize() and upper() string methods. (Tools: Python 3 (Colab / IDE).)

Step-by-step

1. Create the two variables.

```
bs = 'balance sheet'

income = 'income statement'
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

2. Use capitalize() to turn 'balance sheet' into 'Balance sheet', and upper() to turn 'income statement' into 'INCOME STATEMENT', inside one f-string.

```
print(f"The finance statements typically require {bs.capitalize()} and
{income.upper()}.")
```

3. Run the cell and check the output matches exactly, including the full stop.

Check your result

The output is exactly: "The finance statements typically require Balance sheet and INCOME STATEMENT."

Note: This activity is also available as labs/lab-03-format-string.md in the course repository.

Activity 4 — Split and Join Strings

Focus: split a string into parts and join them back.

Goal: Given email = 'finance@company.com', separate the user name and domain name, then join them back together with a '/'.

What you'll build

Code that splits an email address and re-joins the parts with a different separator. (Tools: Python 3 (Colab / IDE).)

Step-by-step

1. Create the email variable.

```
email = 'finance@company.com'
```

2. Split the email at the '@' character into a username and a domain.

```
user, domain = email.split('@')

print(user)

print(domain)
```

3. Join the two parts back with a '/' separator.

```
joined = '/'.join([user, domain])

print(joined)
```

Check your result

The output shows 'finance', then 'company.com', then 'finance/company.com'.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Note: This activity is also available as labs/lab-04-split-and-join-strings.md in the course repository.

Activity 5 — List Operations

Focus: manipulate lists with copy, remove, insert and reverse.

Goal: Given years = ['2017','2018','2020','2021']: copy it to years2, remove '2021' from years2, insert '2019' at position 2, then reverse the order of years2.

What you'll build

A list transformed step by step with the core list methods. (Tools: Python 3 (Colab / IDE).)

Step-by-step

1. Create the original list.

```
years = ['2017', '2018', '2020', '2021']
```

2. Copy the list with copy() — NOT with years2 = years, which would only copy the reference so both names point at the same list.

```
years2 = years.copy()
```

3. Remove '2021' from years2.

```
years2.remove('2021')
```

4. Insert '2019' at index 2 of years2.

```
years2.insert(2, '2019')
```

5. Reverse the order of years2 and print both lists to confirm the original is untouched.

```
years2.reverse()
```

```
print(years)
```

```
print(years2)
```

Check your result

years is unchanged ['2017','2018','2020','2021'] and years2 prints ['2020','2019','2018','2017'].

Note: This activity is also available as labs/lab-05-list-operations.md in the course repository.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Activity 6 — Set — Count Unique Vocabulary

Focus: use a set to find distinct items in text.

Goal: Given the sentence 'A budget is a quantitative plan for acquiring and using resources over a specified period.', find the number of unique vocabulary words.

What you'll build

A set-based word counter that removes duplicate words. (Tools: Python 3 (Colab / IDE).)

Step-by-step

1. Create the sentence variable.

```
sent = "A budget is a quantitative plan for acquiring and using resources over a specified period."
```

2. Normalise the text to lowercase and strip the final full stop, so 'A' and 'a' count as the same word.

```
clean = sent.lower().replace('.', '')
```

3. Split the sentence into a list of words.

```
words = clean.split()
```

4. Convert the list to a set to drop duplicates, then count it with len().

```
unique = set(words)

print(unique)

print('Unique words:', len(unique))
```

Check your result

The duplicate 'a' appears only once in the set, and the unique word count prints 13.

Note: This activity is also available as labs/lab-06-set-count-unique-vocabulary.md in the course repository.

Topic 03 — Operators

Arithmetic · Compound · Comparison · Membership · Logical.

Key concepts

- Arithmetic & compound — + - * / ** % // and their compound forms (+=, -=, …).

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Comparison — < <= > >= == != compare values and return True/False.
- Membership — in and not in test whether an item is inside a sequence.
- Logical — and, or, not combine conditions into business rules.

Activity 7 — Operators

Focus: evaluate comparison and logical operators.

Goal: Given a = 1 and b = 2, predict and then verify: what is (a == 1) and not (b > 3)? Then explore the arithmetic and membership operators.

What you'll build

Verified predictions of Boolean expressions and arithmetic results. (Tools: Python 3 (Colab / IDE).)

Step-by-step

1. Create the variables and write your prediction as a comment first.

```
a = 1

b = 2

# my prediction: ?
```

2. Evaluate the expression — (a == 1) is True, (b > 3) is False, and not False is True; True and True gives True.

```
print((a == 1) and not (b > 3))
```

3. Try the arithmetic operators, including exponent, modulus and floor division.

```
print(2 ** 10)   # exponent

print(7 % 3)     # modulus

print(7 // 3)    # floor division
```

4. Try the compound operators — x += 1 is shorthand for x = x + 1.

```
x = 5

x += 1

x *= 2

print(x)
```

5. Try the membership operators on a list.

```
banks = ['DBS', 'UOB', 'OCBC']

print('DBS' in banks)
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
print('HSBC' not in banks)
```

Check your result

The Boolean expression prints True; 2**10 → 1024, 7%3 → 1, 7//3 → 2, x → 12, and both membership checks print True.

Note: This activity is also available as labs/lab-07-operators.md in the course repository.

Topic 04 — Control Structures, Loop and Comprehension

Conditional · While & For Loops · Multiple Sequences · Comprehension.

Key concepts

- Conditional — if / elif / else route the program down different paths.
- Loops — while repeats until a condition fails; for iterates a sequence.
- Multiple sequences — enumerate() and zip() loop several lists together.
- Comprehension — One-line list / set / dict builders: [transform iteration filter].

Activity 8 — Conditional — Income Classifier

Focus: apply if / elif / else to a business rule.

Goal: Prompt the user for their income and classify it: more than 8000 → 'High income'; 4000 to 8000 → 'Medium high income'; otherwise → 'Average income'.

What you'll build

An interactive income classifier using input() and if/elif/else. (Tools: Python 3 (Colab / IDE).)

Step-by-step

1. Prompt for the income and convert the text input to an integer.

```
income = input('What is your income? ')

income = int(income)
```

2. Write the if / elif / else ladder — test the highest band first.

```
if income > 8000:

    print('High income')

elif income >= 4000:

    print('Medium high income')
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
else:
    print('Average income')
```

3. Run the cell three times with incomes 9000, 5000 and 3000 to exercise every branch.

4. Bonus: rewrite the middle band explicitly as 4000 <= income <= 8000 — Python allows chained comparisons.

```
if 4000 <= income <= 8000:
    print('Medium high income')
```

5. Make the input robust: wrap the prompt in try/except inside a while True loop so a non-numeric entry like 'abc' re-prompts instead of crashing.

```
while True:
    try:
        income = int(input('What is your income? '))
        break
    except ValueError:
        print('Invalid input. Enter a number.')
```

Check your result

9000 → High income, 5000 → Medium high income, 3000 → Average income; typing 'abc' re-prompts instead of crashing.

Note: This activity is also available as labs/lab-08-conditional-income-classifier.md in the course repository.

Activity 9 — Fibonacci Sequence & Golden Ratio (While Loop)

Focus: compute an iterative sequence with a while loop and multiple assignment.

Goal: A Fibonacci number is the sum of the previous two: 1, 1, 2, 3, 5, 8, 13, 21, 34 … Use a while loop to compute the sequence up to 144 and compute the golden ratio.

What you'll build

A while loop producing the Fibonacci sequence and the golden ratio ≈ 1.618. (Tools: Python 3 (Colab / IDE).)

Step-by-step

1. Start with the first two Fibonacci numbers.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
a, b = 1, 1
```

2. Loop while a has not passed 144, printing a and stepping the pair forward with MULTIPLE ASSIGNMENT — both sides are evaluated before assignment.

```
while a <= 144:
    print(a, end=' ')
    a, b = b, a + b
```

3. Compute the golden ratio from the last pair — the ratio of consecutive Fibonacci numbers converges to 1.618…

```
print()
print('Golden ratio ~', b / a)
```

4. Why multiple assignment? Try the same update as two separate lines (a = b then b = a + b) and see the wrong sequence — a has already changed when b is computed.

```
a, b = 1, 1
while a <= 20:
    print(a, end=' ')
    a = b
    b = a + b   # WRONG: uses the NEW a
```

Check your result

The sequence prints 1 1 2 3 5 8 13 21 34 55 89 144 and the golden ratio is approximately 1.618.

Note: This activity is also available as labs/lab-09-fibonacci-sequence-golden-ratio-while-loop.md in the course repository.

Activity 10 — For Loop — Square Numbers

Focus: generate a sequence with for and range().

Goal: Generate a Python list of 10 square numbers [1, 4, 9, 16, 25, 36, 49, 64, 81, 100] using a for loop and range().

What you'll build

A list of the first 10 square numbers built inside a for loop. (Tools: Python 3 (Colab / IDE).)

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Step-by-step

1. Create an empty list to collect the squares.

```
squares = []
```

2. Loop i from 1 to 10 with range(1, 11) and append i squared each time.

```
for i in range(1, 11):

    squares.append(i ** 2)
```

3. Print the finished list.

```
print(squares)
```

4. Check your understanding of range(start, stop, step): what is list(range(2, 12, 2))?

```
print(list(range(2, 12, 2)))
```

Check your result

The list prints [1, 4, 9, 16, 25, 36, 49, 64, 81, 100] and range(2,12,2) gives [2, 4, 6, 8, 10].

Note: This activity is also available as labs/lab-10-for-loop-square-numbers.md in the course repository.

Activity 11 — Loop Multiple Sequences with zip()

Focus: iterate several lists in parallel.

Goal: Given year, revenue and expense lists, fill in the blank so the loop prints each year's revenue and expense together: data = _____; for y, r, e in data: …

What you'll build

A zip() of three lists looped in parallel to print a mini financial report. (Tools: Python 3 (Colab / IDE).)

Step-by-step

1. Create the three lists.

```
year = ['2017', '2018', '2019']

revenue = [150000, 200000, 500000]

expense = [90000, 100000, 250000]
```

2. Fill the blank with zip(year, revenue, expense) — zip pairs the items positionally.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
data = zip(year, revenue, expense)
```

3. Loop and print with an f-string.

```
for y, r, e in data:

    print(f"{y}'s revenue is ${r} and expense is ${e}")
```

4. Bonus: compute the profit inside the loop.

```
for y, r, e in zip(year, revenue, expense):

    print(f'{y} profit: ${r - e}')
```

Check your result

Three lines print, one per year, each showing the matching revenue and expense (e.g. 2017's revenue is $150000 and expense is $90000).

Note: This activity is also available as labs/lab-11-loop-multiple-sequences-with-zip.md in the course repository.

Activity 12 — List Comprehension

Focus: build a filtered list in one line.

Goal: Create a list comprehension of 10 square numbers that are not divisible by 3 and 5: [1, 4, 16, 49, 64, 121, 169, 196, 256, 289]

What you'll build

A one-line comprehension with a transform, an iteration and a filter. (Tools: Python 3 (Colab / IDE).)

Step-by-step

1. Recall the pattern: [transform iteration filter].

2. The transform is x**2; iterate x over a range; filter out x divisible by 3 or 5. Squares of multiples of 3 or 5 are exactly the squares divisible by 3 or 5.

```
a = [x**2 for x in range(1, 18) if x % 3 != 0 and x % 5 != 0]

print(a)

print(len(a))
```

3. Adjust the range until the list holds exactly 10 items — range(1, 18) gives 10 qualifying squares.

Check your result

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

The list prints [1, 4, 16, 49, 64, 121, 169, 196, 256, 289] with length 10.

Note: This activity is also available as labs/lab-12-list-comprehension.md in the course repository.

Topic 05 — Function

Function Syntax · Return Values · Default & Variable Arguments · Lambda, Map, Filter.

Key concepts

- Reusable blocks — def packages logic once; call it everywhere.
- Return values — Functions can return one value — or several at once.
- Flexible arguments — Default, named and variable (*args) arguments.
- Lambda, map, filter — Small anonymous functions that transform and filter data.

Activity 13 — Function — Future Value Calculator

Focus: package a business calculation as a reusable function.

Goal: The FV calculation lets investors predict the profit generated by an investment. Create a function to compute the future value FV = PV × (1 + r)^n for a present value PV, annual interest rate r and n years.

What you'll build

A reusable fv() function returning the future value of an investment. (Tools: Python 3 (Colab / IDE).)

Step-by-step

1. Define the function with three parameters and return the FV formula.

```
def fv(pv, r, n):

    return pv * (1 + r) ** n
```

2. Call it: $10,000 invested at 5% for 10 years.

```
print(fv(10000, 0.05, 10))
```

3. Format the answer as money with an f-string.

```
print(f'Future value: ${fv(10000, 0.05, 10):,.2f}')
```

4. Try other scenarios — change the rate and the years and rerun.

```
print(f'{fv(5000, 0.03, 5):,.2f}')
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Check your result

fv(10000, 0.05, 10) prints approximately 16288.95 — formatted as $16,288.95.

Note: This activity is also available as labs/lab-13-function-future-value-calculator.md in the course repository.

Activity 14 — Multiple Return Values — Discount & Tax

Focus: return more than one value from a function.

Goal: Prompt the user for the order amount, then create a function that returns BOTH the discount amount and the tax for a grocery purchase: discount is 25% if order > 200 else 0; disc_amt = discount*order/100; tax = 7% of the discounted amount.

What you'll build

A function returning (discount amount, tax) unpacked into two variables. (Tools: Python 3 (Colab / IDE).)

Step-by-step

1. Prompt for the order amount and convert it to a float.

```
order = float(input('Order amount? '))
```

2. Define the function computing both values, returned as a pair.

```
def bill(order):
    discount = 25 if order > 200 else 0
    disc_amt = discount * order / 100
    tax = 0.07 * (order - disc_amt)
    return disc_amt, tax
```

3. Call the function and unpack the two return values.

```
disc_amt, tax = bill(order)
print(f'Discount: ${disc_amt:.2f}')
print(f'Tax: ${tax:.2f}')
```

4. Test with an order of 300 (gets the discount) and 100 (no discount).

Check your result

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Order 300 → Discount $75.00 and Tax $15.75; order 100 → Discount $0.00 and Tax $7.00.

Note: This activity is also available as labs/lab-14-multiple-return-values-discount-tax.md in the course repository.

Activity 15 — Variable Arguments — Minimum Finder

Focus: accept any number of arguments with *args.

Goal: Given a variable number of elements, create a function to determine the minimum value. E.g. for 5, 4, 7 the function returns 4. Do it with a loop — no min()!

What you'll build

A my_min(*args) function that scans any number of values for the smallest. (Tools: Python 3 (Colab / IDE).)

Step-by-step

1. Define the function with *args — the arguments arrive as a tuple.

```
def my_min(*args):
    smallest = args[0]
    for x in args:
        if x < smallest:
            smallest = x
    return smallest
```

2. Call it with different numbers of arguments.

```
print(my_min(5, 4, 7))
print(my_min(9, 2, 8, 1, 6))
```

3. Confirm against Python's built-in min().

```
print(min(5, 4, 7))
```

Check your result

my_min(5, 4, 7) prints 4 and my_min(9, 2, 8, 1, 6) prints 1 — matching min().

Note: This activity is also available as labs/lab-15-variable-arguments-minimum-finder.md in the course repository.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Activity 16 — Lambda Function

Focus: trace and write anonymous functions.

Goal: Given f = lambda x, y: 10*x + y and def g(f, x, y=0): return f(x, y) — what is g(f, 4)? Predict first, then verify.

What you'll build

A verified trace of a lambda passed into another function with a default argument. (Tools: Python 3 (Colab / IDE).)

Step-by-step

1. Create the lambda and the function.

```
f = lambda x, y: 10 * x + y


def g(f, x, y=0):
    return f(x, y)
```

2. Predict: g(f, 4) calls f(4, 0) → 10*4 + 0 = 40. Verify it.

```
print(g(f, 4))
```

3. Write your own lambda — cube a number — and test it.

```
cube = lambda x: x ** 3

print(cube(4))
```

Check your result

g(f, 4) prints 40 and cube(4) prints 64.

Note: This activity is also available as labs/lab-16-lambda-function.md in the course repository.

Activity 17 — Map

Focus: transform sequences with map() and lambda.

Goal: Use map() to compute the answers for f = lambda x, y: 10*x + y with x = [0, 1, 2, 3] and y = [2, 4, 6, 8].

What you'll build

A map() over two lists producing [2, 14, 26, 38]. (Tools: Python 3 (Colab / IDE).)

Step-by-step

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

1. Create the lambda and the two lists.

```
f = lambda x, y: 10 * x + y
```

```
x = [0, 1, 2, 3]
```

```
y = [2, 4, 6, 8]
```

2. map() applies f pairwise across both lists; wrap in list() to see the results.

```
result = list(map(f, x, y))
```

```
print(result)
```

3. Check one by hand: the second pair is 10*1 + 4 = 14.

Check your result

The result prints [2, 14, 26, 38].

Note: This activity is also available as labs/lab-17-map.md in the course repository.

Activity 18 — Filter

Focus: select items from a sequence with filter().

Goal: Use filter() to remove all the negative values in x = [0, 1, -4, 2, -6, 3].

What you'll build

A filter() with a lambda that keeps only non-negative numbers. (Tools: Python 3 (Colab / IDE).)

Step-by-step

1. Create the list.

```
x = [0, 1, -4, 2, -6, 3]
```

2. filter() keeps the items where the lambda returns True — keep x >= 0.

```
pos = list(filter(lambda v: v >= 0, x))
```

```
print(pos)
```

3. Bonus: do the same with a list comprehension and compare.

```
print([v for v in x if v >= 0])
```

Check your result

Both approaches print [0, 1, 2, 3] — the negatives are gone.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Note: This activity is also available as labs/lab-18-filter.md in the course repository.

Topic 06 — Modules & Packages

Import Modules · Standard Library · Third-Party Packages · pip · Data & Charts.

Key concepts

- Standard library — math, time, random, datetime — batteries included.
- import — import module, import module as alias, from module import name.
- Third-party packages — pip installs from PyPI: numpy, pandas, matplotlib, yfinance.
- Real finance data — Load CSV / Yahoo Finance data and visualize stock prices.

Activity 19 — Install Third-Party Packages

Focus: install and verify third-party packages with pip.

Goal: Install the course packages on your terminal / CMD prompt (or verify them in Colab): numpy, matplotlib, pandas and yfinance — then import them to confirm.

What you'll build

A working data-analysis toolkit: numpy, pandas, matplotlib and yfinance. (Tools: pip, PyPI, Google Colab / terminal.)

Step-by-step

1. On your terminal / CMD prompt install the four packages (in Colab, prefix with ! — numpy, pandas and matplotlib are pre-installed there).

```
pip install numpy
```

```
pip install matplotlib
```

```
pip install pandas
```

```
pip install yfinance
```

2. Verify each package imports and print its version.

```
import numpy, pandas, matplotlib
```

```
import yfinance
```

```
print(numpy.__version__)
```

```
print(pandas.__version__)
```

3. Browse https://pypi.org — every third-party Python package lives here.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

4. Optional: the Anaconda distribution (anaconda.com) bundles 150+ scientific packages pre-built — NumPy, Pandas, SciPy, Matplotlib, IPython and more.

Check your result

All four packages import without ModuleNotFoundError and print a version number.

Note: This activity is also available as labs/lab-19-install-third-party-packages.md in the course repository.

Activity 20 — Import & Visualize Stock Data from CSV

Focus: load a CSV with pandas and chart it with matplotlib.

Goal: Import historical stock data from a CSV file with pandas, select the closing price and visualize the price history with matplotlib.

What you'll build

A line chart of the Singtel closing-price history from a CSV file. (Tools: pandas, matplotlib, CSV (labs/data/singtel.csv).)

Step-by-step

1. Import pandas and load the CSV (upload labs/data/singtel.csv to Colab, or use its path in your IDE).

```
import pandas as pd

stock = pd.read_csv('singtel.csv')

print(stock.head())
```

2. Select the closing-price column.

```
stock_close = stock[['Close']]
```

3. Plot the price history with matplotlib — line plus point markers.

```
import matplotlib.pyplot as plt

plt.figure(figsize=(14, 5))

plt.plot(stock_close, 'b')

plt.plot(stock_close, 'ro')

plt.title('Singtel Close Price Representation')

plt.xlabel('Trading Days')

plt.ylabel('Close Price')
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
plt.show()
```

4. Read the chart: where were the highest and lowest closes?

Check your result

head() shows the CSV columns (Date, Open, High, Low, Close) and a blue line chart with red markers appears, titled 'Singtel Close Price Representation'.

Note: This activity is also available as labs/lab-20-import-visualize-stock-data-from-csv.md in the course repository.

Activity 21 — Import Data from Yahoo Finance

Focus: download live market data and plot it.

Goal: Use the yfinance package to download Singtel (Z74.SI) stock data from Yahoo Finance for a date range, then plot the closing price.

What you'll build

A live Yahoo Finance download and a closing-price chart. (Tools: yfinance, pandas, matplotlib.)

Step-by-step

1. Import yfinance and download one year of Singtel data.

```
import yfinance as yf

stock = yf.download('Z74.SI', start='2024-01-01', end='2024-12-31')

print(stock.tail())
```

2. Select the closing price.

```
stock_close = stock[['Close']]
```

3. Plot it, with title and axis labels.

```
import matplotlib.pyplot as plt

plt.figure(figsize=(14, 5))

plt.plot(stock_close, 'b')

plt.plot(stock_close, 'ro')

plt.title('Singtel Close Price Representation')

plt.xlabel('Trading Days')
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
plt.ylabel('Close Price')

plt.show()
```

4. Try another ticker — e.g. D05.SI (DBS) or AAPL — and rerun.

```
stock = yf.download('D05.SI', start='2024-01-01', end='2024-12-31')
```

Check your result

tail() shows recent trading rows and the chart plots a year of closing prices; swapping the ticker redraws the chart for the new stock.

Note: This activity is also available as labs/lab-21-import-data-from-yahoo-finance.md in the course repository.

Keep Practising

- Redo every activity from a blank notebook until you can write it without the guide.
- Change the data — different numbers, strings, tickers — and predict the output before running.
- Try the recommended follow-on courses to advance: Python Data Analysis Masterclass; AI Vibe Coding with Python; SQL Masterclass.
- Python documentation: https://docs.python.org/3/ · Package index: https://pypi.org

Glossary

- **Variable —** A name bound to a value; created by assignment (x = 5).
- **Data type —** The kind of value: int, float, complex, str, list, tuple, dict, set.
- **Mutable / immutable —** Whether a value can change in place — lists are mutable; tuples and strings are not.
- **f-string —** A string literal prefixed with f that embeds expressions: f"{amt:.2f}".
- **Slicing —** Selecting part of a sequence with [start:stop:step].
- **Comprehension —** A one-line builder for a list/set/dict: [transform iteration filter].
- **Function —** A reusable block defined with def that may accept arguments and return values.
- **Lambda —** A small anonymous function: lambda x, y: 10*x + y.
- **map() / filter() —** Apply a function to every item / keep the items where a function is True.
- **Module / package —** A Python file / folder of modules you import; install third-party packages with pip.
- **pip —** Python's package installer, pulling from the Python Package Index (PyPI).
- **pandas / matplotlib / yfinance —** Data analysis, charting and Yahoo Finance download packages used in Topic 6.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg